SENSOR 接入指南

1.0.1

2022-09-07

上海富瀚微电子股份有限公司
地址：上海市徐汇区宜山路 717 号 2 号楼 5 楼-6 楼邮编：200233
网址：http://www.fullhan.com
传真：021-64066786
电话：021-61121558
邮箱：sales@fullhan.com

版权所有 © 上海富瀚微电子股份有限公司。保留一切权利。
非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标申明

Fullhan®、富瀚均为上海富瀚微电子股份有限公司的商标

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

目录

目录 1

1 前言 1-1

1.1 修订记录 1-1

1.2 适用范围 1-1

2 概述 2-1

3 Sensor 接口 3-1

3.1 接口函数 3-1

3.2 MIPI 接口说明 3-7

3.3 DVP 接口说明 3-9

3.4 软件流程 3-9

3.5 代码示例 3-10

3.5.1 文件结构 3-10

3.5.2 创建 sensor 3-11

3.5.3 初始化流程 3-12

4 常见问题 Q&A 4-1

4.1 如何配置 sensor 幅面 4-1

4.1.1 参数解释 4-1

4.1.2 约束条件 4-1

4.1.3 示意图 4-2

4.2 启动时为何出现无法读写 sensor 的错误 4-2

4.3 如何检测配置的幅面是否正确 4-3

4.4 如何实时切换帧率 4-4

4.5 如何实时切换分辨率 4-5

4.6 实时切换制式时为何出现无 pic_end 中断错误 4-6

4.7 如何提高 sensor 启动速度 4-6

4.8 如何选择 sensor 输入时钟 4-6
4.9 如何抓取 sensor 的 raw 数据 4-6
4.10 如何导出 hex 文件 4-8

 版权所有 @ 上海富瀚微电子股份有限公司

1 前言

本文档主要描述了如何在 FULLHAN 平台下接入 sensor。

1.1 修订记录

修订时间	修订版本	修订人	修订描述
2021-09-15	V1.0.0	马金石	继承 FH885xV200 文档，初稿
2022-09-07	V1.0.1	徐磊	芯片增加 FH8626V200

1.2 适用范围

本文档适用于以下芯片

- FH8636
- FH8852V201
- FH8852V202
- FH8626V200

版权所有 @ 上海富瀚微电子股份有限公司

2 概述

ISP 的固件主要包括系统控制单元，算法策略单元，sensor 单元及调试单元。系统控制单元主要用于处理中断，提供统计信息等；算法策略单元是 ISP 的核心，用于调整图像的整体效果，主要包括自动曝光，自动白平衡，色彩校正，去噪，锐化，坏点校正，宽动态，镜头阴影校正等；sensor 单元用于接入不同的 sensor；调试单元主要实现采用外部调试工具对寄存器进行读写的功能。

本文档主要描述了如何在 FULLHAN 平台下接入 sensor 的过程，在接入 sensor 的时候，需要按照下文描述的内容实现 FULLHAN sensor 相关的接口函数，最后运行相应的程序。在接入过程中遇到的一些常见问题，也在后面列了出来。

版权所有 @ 上海富瀚微电子股份有限公司

3 Sensor 接口

Sensor 需要向 ISP 库注册回调函数，用于实现与 ISP 的对接。每一个 sensor 都需要向 ISP 库注册回调函数，如 sensor 的初始化，曝光时间的控制，模拟增益的控制等等，ISP 通过这些回调函数实现对 sensor 的控制。

3.1 接口函数

Sensor 回调函数的数据结构如下：

```
struct isp_sensor_if
{
    FH_CHAR*     name;
    FH_SINT32    (*get_vi_attr)(ISP_VI_ATTR_S *vi_attr);
    FH_SINT32    (*set_flipmirror)(FH_UINT32 sensor_en_stat);
    FH_SINT32    (*get_flipmirror)(FH_UINT32 *sensor_en_stat);
    FH_SINT32    (*set_iris)(FH_UINT32 iris);
    FH_SINT32    (*init)(FH_VOID);
    FH_VOID      (*reset)(FH_VOID);
    FH_SINT32    (*deinit)(FH_VOID);
    FH_SINT32    (*set_sns_fmt)(int format);
    FH_SINT32    (*kick)(FH_VOID);
    FH_SINT32    (*set_sns_reg)(FH_UINT16 addr, FH_UINT16 data);
    FH_SINT32    (*set_exposure_ratio)(FH_UINT32 exposure_ratio);
    FH_SINT32    (*get_exposure_ratio)(FH_UINT32 *exposure_ratio);
    FH_SINT32    (*get_sensor_attribute)(char * name,FH_UINT32 *value);
    FH_SINT32    (*get_sns_reg)(FH_UINT16 addr, FH_UINT16 *data);
    FH_SINT32    (*get_awb_gain)(FH_UINT32 *awb_gain);
    FH_SINT32    (*set_awb_gain)(FH_UINT32 *awb_gain);
    SENSOR_USER_DEFINED_DATA*    data;
    FH_SINT32    (*common_if)(unsigned int cmd, ISP_SENSOR_COMMON_CMD_DATA0* data0, I
    FH_SINT32    (*get_sns_ae_default)(Sensor_AE_Default_S *sensAEDefault);
    FH_SINT32    (*get_sns_ae_info)(Sensor_AE_INFO_S *sensAEInfo);
    FH_SINT32    (*set_sns_intt)(FH_UINT32 intt, FH_UINT8 fNo);
    FH_SINT32    (*calc_sns_valid_intt)(FH_UINT32 *u32Intt);
    FH_SINT32    (*set_sns_gain)(FH_UINT32 again, FH_UINT8 fNo);
    FH_SINT32    (*calc_sns_valid_gain)(FH_UINT32 *u32Gain);
    FH_SINT32    (*set_sns_frame_height)(FH_UINT32 frameH);
    FH_UINT32*   (*get_sensor_mirror_flip_bayer_format)(void);
```


版权所有 @ 上海富瀚微电子股份有限公司

```
    FH_UINT32*  (*get_user_awb_gain)(FH_UINT32 idx);
    FH_UINT32*  (*get_user_ltm_curve)(FH_UINT32 idx);
    FH_SINT32   (*is_sensor_connect)(void);

};
```

各函数的意义如下：

- name

  sensor 的名字，定义为一个字符串

- get_vi_attr(ISP_VI_ATTR_S *vi_attr)

  获取 sensor 的幅面等信息，结构体 ISP_VI_ATTR_S 的定义如下

```
typedef struct _ISP_VI_ATTR_
{
    FH_UINT16      u16WndHeight;   // sensor 幅面高度
    FH_UINT16      u16WndWidth;    // sensor 幅面宽度
    FH_UINT16      u16InputHeight; // sensor 输入图像高度
    FH_UINT16      u16InputWidth;  // sensor 输入图像宽度
    FH_UINT16      u16OffsetX;     // 裁剪水平偏移
    FH_UINT16      u16OffsetY;     // 裁剪垂直偏移
    FH_UINT16      u16PicHeight;   // 处理的图像高度
    FH_UINT16      u16PicWidth;    // 处理的图像宽度
    FH_UINT16      u16FrameRate;   // 帧率
    ISP_BAYER_TYPE enBayerType;    // bayer 数据格式
} ISP_VI_ATTR_S;
```

- set_flipmirror(FH_UINT32 sensor_en_stat)

  配置 sensor 的水平翻转和镜像。传入参数

sensor_en_stat	水平翻转	镜像
0x1	是	否
0x2	否	是
0x3	是	是

- get_flipmirror(FH_UINT32 *sensor_en_stat)

  获取当前图像水平镜像及垂直翻转的状态，获取的值的对照值按上表。

 版权所有 @ 上海富瀚微电子股份有限公司

- set_iris(FH_UINT32 iris)

  设置光圈大小

- init

  初始化 sensor，主要包括注册 sensor 设备号，重置 sensor，配置一些 sensor 的初始化寄存器

- reset

  复位 sensor

- deinit

  注销 sensor 的回调函数，各回调函数指针设为 NULL

- set_sns_fmt(int format)

  配置 sensor 的输出制式，参数为 format，是枚举变量，具体定义如下：

```
enum FORMAT
{
    FORMAT_720P25        = 0x03,
    FORMAT_720P30        = 0x04,
    FORMAT_960P25        = 0x0103,
    FORMAT_960P30        = 0x0104,
    FORMAT_1080P15       = 0x0201,
    FORMAT_1080P20       = 0x0202,
    FORMAT_1080P25       = 0x0203,
    FORMAT_1080P30       = 0x0204,
    FORMAT_1080P60       = 0x0207,
    FORMAT_1200P30       = 0x0304,
    FORMAT_1536X1536P15 = 0x0401,
    FORMAT_1536X1536P25 = 0x0403,
    FORMAT_1536X1536P30 = 0x0404,
    FORMAT_1536P15       = 0x0501,
    FORMAT_1536P25       = 0x0503,
    FORMAT_1536P30       = 0x0504,
    FORMAT_1536P40       = 0x0505,
    FORMAT_400WP15       = 0x0601,
    FORMAT_400WP20       = 0x0602,
    FORMAT_400WP25       = 0x0603,
    FORMAT_400WP30       = 0x0604,
    FORMAT_500W12P5      = 0x0700,
```


版权所有 @ 上海富瀚微电子股份有限公司

```
    FORMAT_500WP15       = 0x0701,
    FORMAT_500WP20       = 0x0702,
    FORMAT_800WP15       = 0x0801,
    FORMAT_270P240       = 0x0909,
    FORMAT_540P120       = 0x0a08,
    FORMAT_540P180       = 0x0a0c,
    FORMAT_2304X1296P25 = 0x0b03,
    FORMAT_192X192P220  = 0x0c0a,
    FORMAT_384X384P110  = 0x0d0b,
    FORMAT_WDR_1080P25  = 0x10203,
    FORMAT_WDR_1080P30  = 0x10204,
    FORMAT_WDR_1536P15  = 0x10501,
    FORMAT_WDR_1536P25  = 0x10503,
    FORMAT_WDR_1536P30  = 0x10504,
    FORMAT_WDR_1536P40  = 0x10505,
    FORMAT_WDR_400WP15  = 0x10601,
    FORMAT_WDR_400WP20  = 0x10602,
    FORMAT_WDR_400WP25  = 0x10603,
    FORMAT_WDR_400WP30  = 0x10604,
    FORMAT_WDR_500W12P5 = 0x10700,
    FORMAT_WDR_500WP15  = 0x10701,
    FORMAT_WDR_500WP20  = 0x10702,
    FORMAT_WDR_800WP15  = 0x10801,
    SENSOR_FORMAT_MAX   = 0xfffff,
};
```

- kick

  部分 sensor 在下载完配置之后，需要配置 kick 寄存器才能够正常启动。

- set_sns_reg(FH_UINT16 addr, FH_UINT16 data)

  往 senor 的指定地址，写入指定的数据。

- (set_exposure_ratio)(FH_UINT32 exposure_ratio)

  wdr 下配置长短帧曝光比例，wdr 策略中会传入该参数

- (get_exposure_ratio)(FH_UINT32 *exposure_ratio)

  wdr 下获取长短帧曝光比例

- get_sensor_attribute(char* name,FH_UINT32 *value)

  获取指定 name 的状态值

版权所有 @ 上海富瀚微电子股份有限公司

- FH_SINT32 (*get_sns_reg)(FH_UINT16 addr, FH_UINT16* data)

  获取 sensor 指定寄存器的值

- FH_SINT32 (*get_awb_gain)(FH_UINT32* awb_gain)

  获取取到的是 sensor 当前配置的 awb 增益，仅支持 awb 增益的 sensor。

- FH_SINT32 (*set_awb_gain)(FH_UINT32* awb_gain);

  配置 sensor 的 awb 增益值，仅支持 awb 增益的 sensor.

- FH_SINT32 (*common_if)(unsigned int cmd, ISP_SENSOR_COMMON_CMD_DATA0* data0, ISP_SENSOR_COMMON_CMD_DATA1* data1);

  自定义接口，通过 cmd 可以控制底层逻辑，支持传入的参数最大 128 个字的大小。

```
typedef union
{
    unsigned int dw[128];
} SENSOR_USER_DEFINED_DATA;
```

- FH_SINT32 (*get_sns_ae_default)(Sensor_AE_Default_S* sensAEDefault)

  获取 AE 曝光 intt、again 最大最小值的限制，框架高度和最小复位行数。结构体 Sensor_AE_Default_S 的定义如下。

```
typedef struct _Sensor_AE_Default_S
{
    unsigned int  minIntt;  // 最小曝光时间，单位为行
    unsigned int  maxIntt;  // 最大曝光时间，单位为行
    unsigned int  minAGain;  // 最小曝光增益,U.6 精度
    unsigned int  maxAGain;  // 最大曝光增益,U.6 精度
    unsigned int  fullLineStd;  // 框架高度
    unsigned int  MinRstLine;  // 计算最大曝光时间时的最小复位行数
}Sensor_AE_Default_S;
```


版权所有 @ 上海富瀚微电子股份有限公司

- FH_SINT32 (*get_sns_ae_info)(Sensor_AE_INFO_S* sensAEInfo)

  获取当前曝光状态参数，intt、again、行频、框架高度，结构体 Sensor_AE_INFO_S 如下。

```
typedef struct _Sensor_AE_INFO_S
{
    unsigned int  currIntt;  // 当前曝光时间，单位为行
    unsigned int  currAGain;  // 当前曝光增益，u.6 精度
    unsigned int  currHsf;  // 当前行频
    unsigned int  currFrameH;  // 当前框架高度
}Sensor_AE_INFO_S;
```

- FH_SINT32 (*set_sns_intt)(FH_UINT32 intt, FH_UINT8 fNo)

  配置 senor 的曝光行数，fNo 在宽动态模式下使用，表示一帧当中不同的帧，长帧、中帧、短帧、超短帧等。

- FH_SINT32 (*calc_sns_valid_intt)(FH_UINT32 u32Intt, FH_UINT32* u32VIntt)

  根据传入的曝光行数，返回实际 sensor 能够支持的曝光行数，对曝光行数有限制的 sensor 需要实现该函数。

- FH_SINT32 (*set_sns_gain)(FH_UINT32 again, FH_UINT8 fNo)

  配置 senor 的模拟增益，fNo 在宽动态模式下使用，表示一帧当中不同的帧，长帧、中帧、短帧、超短帧等。

- FH_SINT32 (*calc_sns_valid_gain)(FH_UINT32 u32Gain, FH_UINT32* u32VGain)

  根据传入的模拟增益，返回实际 sensor 能够支持的模拟增益，对模拟增益有限制的 sensor 需要实现该函数。

  FH_SINT32 (*is_sensor_connect)(void);

- FH_SINT32 (*set_sns_frame_height)(FH_UINT32 frameH)

  配置 sensor 的框架高度，支持慢快門的 sensor 需要实现该函数。frameH 的值，单位是行数，即目标配置的框架高度。

版权所有 @ 上海富瀚微电子股份有限公司

- FH_UINT32* (*get_sensor_mirror_flip_bayer_format)(void)

  获取 sensor 在镜像、水平翻转之后的 bayer 的排列方式。

- FH_UINT32* (*get_user_awb_gain)(FH_UINT32 idx)

  获取用户自定义的 awb 增益。

- FH_UINT32* (*get_user_ltm_curve)(FH_UINT32 idx)

  获取用户自定义的 ltm 曲线。

- FH_SINT32 (*is_sensor_connect)(void)

  RTT 下多 sensor 功能的时候，判断 sensor 是否连接。

注意：

1. 初始化 sensor 时一定要注册 sensor 设备号，否则无法配置 sensor；
2. set_sns_fmt 一定要在初始化 sensor 之后，即 API_ISP_SetSensorFmt 要在 API_ISP_SensorInit 之后；
3. sensor 一些初始化寄存器的通用配置可以放入 init 中，也可以都放入 set_sns_fmt 中；
4. init 函数只能在上电的时候调用一次，若运行过程中重复调用，则会出现设备重复打开的错误；
5. 以上的各个回调函数可以根据需要选择实现，不一定每个 sensor 都需要实现所有的函数。

3.2 MIPI 接口说明

在接入 mipi sensor 的过程中，需要在 Sensor_Init 函数中调用 mipi 相关的函数，对 sensor 进行初始化。

1. 初始化的函数

   void mipi_init(struct mipi_conf *conf)

版权所有 @ 上海富瀚微电子股份有限公司

2. 结构体的说明

struct mipi_conf 的数据结构如下：

```
struct mipi_conf
{
    int frq_range;
    int sensor_mode;
    int raw_type;
    int lf_vc_id;
    int sf_vc_id;
};
```

数据定义如下：

- frq_range：mipi 每条 lane 的时钟频率
- sensor_mode：用于选择 sensor 的类型，是否是 sony 的，是否是 wdr，sensor_mode 的枚举变量如下：

```
enum SENSOR_MODE
{
    NON_WDR = 0;
    NOT_SONY_WDR_USE_WDR = 1,
    SONY_WDR_NOT_USE_WDR = 2,
    SONY_WDR_USE_WDR = 3,
};
```

- raw_type：选择数据进来的位宽，10bit 还是 12bit，raw_type 的枚举变量如下：

```
enum RAW_TYPE
{
    RAW10 = 0,
    RAW12 = 1,
};
```


版权所有 @ 上海富瀚微电子股份有限公司

- lf_vc_id：长帧虚拟通道 id，每个 sensor 可能不一样，根据 sensor 文档定。
- sf_vc_id：短帧虚拟通道 id，也需参考 sensor 文档。

3. 示例代码

下面以将 mipi 初始化为宽动态的模式举例。

```
struct mipi_conf config;
config.frq_range = R_800_849;
config.sensor_mode = NOT_SONY_WDR_USE_WDR;
config.raw_type = RAW10;
config.lane_num = 4;

config.lf_vc_id = 0;
config.sf_vc_id = 1;

mipi_init(&config);
```

3.3 DVP 接口说明

```
支持接入并口 sensor，接口并口的时候相比于接入 mipi sensor 不需要初始化一堆接口.
```

3.4 软件流程

Sensor 的启动流程如图 2-1 所示。主要分为以下几个步骤

1. 创建 sensor，根据需要实现 sensor 的接口函数。
2. 注册 sensor 至 ISP，调用 API_ISP_SensorRegCb 接口。
3. 初始化 sensor，调用 API_ISP_SensorInit 接口。
4. 配置 sensor 输出制式，调用 API_ISP_SetSensorFmt 接口。

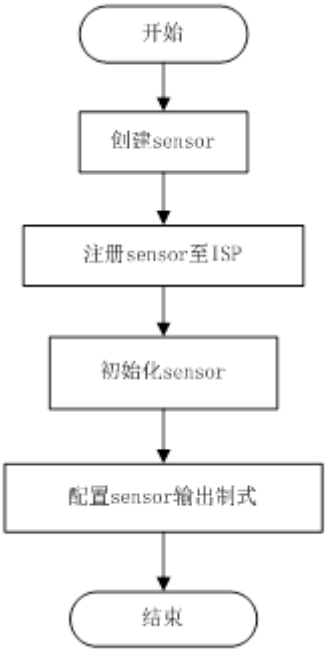

Figure 1: sensor 初始化流程

3.5 代码示例

3.5.1 文件结构

若要新接入一款 sensor，一般所要创建的文件有：

- xxx.h

  表示 sensor 相关的头文件，sensor 的名称以 xxx 表示，下同。

- xxx.c
- makefile
- hex 文件（保存白天及晚上的效果参数）

需要用到的头文件有：

- isp_sensor_if.h
- sensor_ops.h
- isp_enum.h

 版权所有 @ 上海富瀚微电子股份有限公司

3.5.2 创建 sensor

以 OVOS05 为例：

```
#define SENSOR_OVOS05_MIPI    "ovos05_mipi"
#ifdef OS_RTT
struct isp_sensor_if ovos05_mipi_sensor_if __attribute__((section(".sensor_drv"))) =
#else
struct isp_sensor_if ovos05_mipi_sensor_if = {
#endif
    .name = SENSOR_OVOS05_MIPI,
    .get_vi_attr = GetSensorViAttr,
    .set_flipmirror = SetSensorFlipMirror,
    .get_flipmirror = GetSensorFlipMirror,
    .set_iris = SetSensorIris,
    .init = Sensor_Init,
    .reset = SensorReset,
    .deinit = Sensor_DeInit,
    .set_sns_fmt = SetSensorFmt,
    .kick = Sensor_Kick,
    .set_sns_reg = SetSensorReg,
    .set_exposure_ratio = SetExposureRatio,
    .get_exposure_ratio = GetExposureRatio,
    .get_sensor_attribute = GetSensorAttribute,
    .set_lane_num_max = SetLaneNumMax,
    .get_sns_reg = GetSensorReg,
    .get_awb_gain = GetSensorAwbGain,
    .set_awb_gain = SetSensorAwbGain,
    .data = 0,
    .common_if = SensorCommonIf,
    .get_sns_ae_default = GetAEDefault,
    .get_sns_ae_info = GetAEInfo,
    .set_sns_intt  = SetIntt,
    .calc_sns_valid_intt = CalcSnsValidIntt,
    .set_sns_gain = SetGain,
    .calc_sns_valid_gain = CalcSnsValidGain,
    .set_sns_frame_height = SetSnsFrameH,
    .get_sensor_mirror_flip_bayer_format = GetMirrorFlipBayerFormat,
    .get_user_awb_gain = GetUserSensorAwbGain,
    .get_user_ltm_curve = GetSensorLtmCurve,
    .is_sensor_connect = Sensor_Isconnect,
};

#ifdef OS_RTT
#else
struct isp_sensor_if* (Sensor_Create)() {
    return &ovos05_mipi_sensor_if;
```

 版权所有 @ 上海富瀚微电子股份有限公司

```
}

void (Sensor_Destroy)(struct isp_sensor_if* s_if) {
    memset(&ovos05_mipi_sensor_if, 0, sizeof(struct isp_sensor_if));
}
#endif
```

等号左边是 sensor 接口函数指针，等号右边是 ovos05 各个接口函数的具体实现。

3.5.3 初始化流程

sensor 的初始化流程示例代码为：

```
    struct isp_sensor_if *sensor;
    int format;

    sensor = Sensor_Create();

    API_ISP_SensorRegCb(0, sensor);

    API_ISP_SensorInit();

    API_ISP_SetSensorFmt(format);
```


版权所有 @ 上海富瀚微电子股份有限公司

4 常见问题 Q&A

4.1 如何配置 sensor 幅面

对于 isp 中对应的 ISP_VI_ATTR_S 的配置，以 960p25 为例，如下：

```
u16WndWidth = 1600;
u16WndHeight = 1350;
u16InputWidth = 1280;
u16InputHeight = 960;
u16PicWidth = 1280;
u16PicHeight = 960;
u16OffsetX = 0;
u16OffsetY = 0;
```

4.1.1 参数解释

- u16WndWidth 图像框架幅面的宽度
- u16WndHeight 图像框架幅面的高度
- u16InputWidth 图像有效幅面的宽度
- u16InputHeight 图像有效幅面的高度
- u16PicWidth ISP 实际处理的图像宽度
- u16PicHeight ISP 实际处理的高度
- u16OffsetX 实际处理图像相对于图像有效幅面的水平方向的偏移位置
- u16OffsetY 实际处理图像相对于图像有效幅面的垂直方向的偏移位置

4.1.2 约束条件

- u16InputWidth >= u16PicWidth + u16OffsetX
- u16InputHeight >= u16PicHeight + u16OffsetY

 版权所有 @ 上海富瀚微电子股份有限公司

- u16WndWidth > u16PicWidth
- u16WndHeight > u16PicHeight

4.1.3 示意图

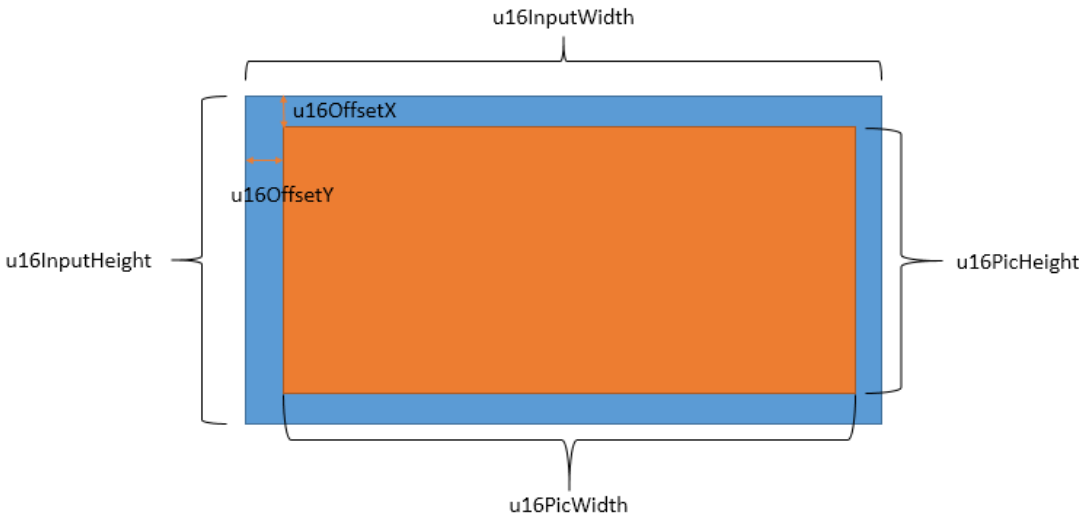

Figure 2: 各个参数关系示意图

4.2 启动时为何出现无法读写 sensor 的错误

客户在启动过程中出现

```
"ERROR: Unable to write sensor register!"
```

的打印信息。一般情况下，若遇到该问题，很有可能是 sensor 的设备号及模式初始化错误。

Sensor 设备初始化函数为 SensorDevice_Init(FH_UINT16 i2c_devid, FH_UINT8 mode)，第一个参数为设备号，由 sensor 厂商提供，写入时需右移一位，例如 IMX138，sony 提供的设备号为 0x34，则 i2c_devid = 0x1a；第二个参数为模式，对于 I2C 通信接口，范围为 0~3，定义如下：

Mode 0：8bit 地址，8bit 数据

Mode 1：8bit 地址，16bit 数据

Mode 2：16bit 地址，8bit 数据

Mode 3：16bit 地址，16bit 数据

4.3 如何检测配置的幅面是否正确

当配置完 sensor 运行时，若发现图像显示不正常，发生移位，花屏等现象，或者完全没有图像，此时应该先检查 sensor 输出的幅面是否与预期一致。硬件中有专门的寄存器存放了当前检测出的图像的框架幅面及有效幅面，用户在使用的时候可以使用 FULLHAN 提供的调试接口。

方法一：

通过命令行输入相应的指令，可以查看对应的调试信息。

1. linux 下，在运行完相应的 demo 程序之后，通过网络连接上设备后，按照下图命令进行操作，可以查看到相应的信息。

```
fh-linux# cat /proc/driver/isp

ISP Status

CTRL:
        SDK VERSION :           v0.0.0(g0000000)@
        Driver Version:         ge7e5c65-dirty
        Sensor FPS:             10000/399(25.00)
        ISP Resolution:         1920x1080
        Detect frame size:      6667x1199
        Detect input size:      1920x1080
        ISPP PIC_START Count:   2816
        ISPP PIC_END Count:     2815
        IPF PIC_START Count:    2815
        IPF PIC_END Count:      2815
        IPB PIC_START Count:    2815
        IPB PIC_END Count:      2814
```

 版权所有 @ 上海富瀚微电子股份有限公司

```
        ISPP OVERFLOW Count:    0
        ISPP CIRCUALBUFF:       OFF
        VPU BUSY Count:         0
        AE STAT ERR Count:      0
```

2. RTT 下使用以下的命令查看相应的信息

```
finsh> isp_read_proc

ISP Status

CTRL:
        SDK VERSION :           v0.0.0(g0000000)@
        Driver Version:         ge7e5c65-dirty
        Sensor FPS:             10000/399(25.00)
        ISP Resolution:         1920x1080
        Detect frame size:      6667x1199
        Detect input size:      1920x1080
        ISPP PIC_START Count:   2816
        ISPP PIC_END Count:     2815
        IPF PIC_START Count:    2815
        IPF PIC_END Count:      2815
        IPB PIC_START Count:    2815
        IPB PIC_END Count:      2814
        ISPP OVERFLOW Count:    0
        ISPP CIRCUALBUFF:       OFF
        VPU BUSY Count:         0
        AE STAT ERR Count:      0
```

方法二：调用 API_ISP_DetectPicSize 函数，若返回值为 0，则说明检测到的幅面与所配置的一致，若非 0，则不一致。可以简单的定位幅面与预期是否符合，详细的调试推荐使用上面提到的方法一。

4.4 如何实时切换帧率

以 IMX138 为例，假设当前输出为 960p25，需要切换至 960p30，可以通过改变图像的框架高度，从原来的 1350 变成 1125。需要注意的是，在改变 sensor 的框架高

 版权所有 @ 上海富瀚微电子股份有限公司

度前，需停止 ISP，待所有配置都完成后，再重新运行 ISP，虽然只是切换了帧率，输出的有效幅面不变，但在修改时还是需要先停止 VPU，完成后再重新使能。

参考代码如下：

```
ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Disable());

API_ISP_Pause(); //停止 ISP

switch(format)
{

    case FORMAT_960P25:
        format = FORMAT_960P30;
        break;
    case FORMAT_960P30:
        format = FORMAT_960P25;
    break;

}

API_ISP_SetSensorFmt(format);

API_ISP_Resume(); //运行 ISP

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Enable(0));
```

4.5 如何实时切换分辨率

以 IMX138 为例，假设当前输出为 960p25，需要切换至 720p25，两种分辨率的框架幅面是一致的，有效幅面需要改变，从 1280*960* 切换至 *1280* 720，ISP 切换的流程与上一小节一致，但 VPSS 处理的幅面需要相应修订，同样，需在 VPSS disable 的状态下进行配置，配置完成后再 enable。

VPSS 的参考代码如下：

```
FH_VPU_SIZE vi_size;

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Disable());
```

```
vi_size.vi_size.u32Width = 1280;

vi_size.vi_size.u32Height = 720;

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_SetViAttr(&vi_size));

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Enable(0));
```

4.6 实时切换制式时为何出现无 pic_end 中断错误

若发现上电直接运行 960p 或者 720p 时，图像是正常的，但在运行过程中直接切换分辨率或者帧率的时候出现 “wait pic_end fail” 的打印信息，则有可能是时序上出了问题，建议可以在切换幅面时多加一些等待时间。

4.7 如何提高 sensor 启动速度

由于和 sensor 之间的通信采用 I2C 接口，I2C 的速率较慢，一般在 400kb/s，开机启动很大一部分时间是花在 sensor 寄存器的初始化上的，为了确保写入的正确性，每次写完一个 sensor 寄存器后，会等待一段时间，内核系统中自带的函数是 usleep，单位为 1μm，所以 usleep(1000) 即为 1ms，当发现启动速度非常慢时，可以适当减小其延时，一般可采用 usleep(300) 左右。

4.8 如何选择 sensor 输入时钟

调用 API_ISP_SetCisClk 接口可以实现 sensor 输入时钟的切换，需要注意的是必须在 API_ISP_Init 后使用，否则会出错。

4.9 如何抓取 sensor 的 raw 数据

当图像出现一些异常效果时，有时需要确认是 sensor 的输出源存在问题，还是后续的图像处理导致，这时就需要抓取 sensor 的 raw 数据。

版权所有 @ 上海富瀚微电子股份有限公司

API 接口中的 API_ISP_GetRaw 函数用于实现该功能。

函数原型为：

```
FH_SINT32 API_ISP_GetRaw(FH_VOID* pRawBuff, FH_UINT32 u32Size,
 FH_UINT32 u32FrameCnt);
```

其中 pRawBuff 是用户分配的用于存放数据的内存，u32Size 是分配的内存大小，u32FrameCnt 是需要抓取的帧数。

示例代码如下：

```
FILE* rawfile;
unsigned char* rawbuff;
ISP_VI_ATTR_S vi_attr;
int raw_size;
char filename[]="raw_data.raw";

framecount=1;

rawfile = fopen(filename, "wb");

if (NULL == rawfile){
    printf("[ERROR]: cannot create file - %sn", filename);
    return -1;
}

API_ISP_GetViAttr(&vi_attr);

raw_size = vi_attr.u16PicWidth*vi_attr.u16PicHeight*12/8;

rawbuff = (unsigned char*)malloc(raw_size * framecount);

if (NULL == rawbuff){
    printf("[ERROR]: not enough memory for dump raw data n");
    return -1;
}

API_ISP_GetRaw(rawbuff, raw_size * framecount, framecount );

fwrite(rawbuff, 1, raw_size * framecount, rawfile);

free(rawbuff);
```


版权所有 @ 上海富瀚微电子股份有限公司

```
    fclose(rawfile);
```

注意：由于抓取一帧数据的时间大于一帧，所以当抓取的帧数大于 1 时，不能实现连续抓取，中间是跳帧的。

4.10 如何导出 hex 文件

当调试完成后，需要将 DRV 寄存器保存成 hex 文件，具体步骤如下：

1、coolview 选择工具-数据读写，参数配置界面如下：

Figure 3: 导出参数界面示意图

2、保存之后，用 ultraEdit（或其他二进制文件读取软件）打开，将文件剪裁成 isp_param 结构体大小；

针对白天场景调试，导出白天的 hex 文件；然后再针对晚上切黑白的场景进行调试，导出晚上的 hex 文件。

版权所有 @ 上海富瀚微电子股份有限公司